

deixis: A Minimal Floor for Keyed Tree Structure

Identity, pointing, and readings over one container

Julian Matschinske julian@matschinske.com

September 2026 — draft v4.1, mandatory node values (ADR 0010)

Abstract

deixis is a specification of finite byte-keyed trees over one opaque slot: $\text{Node}(T) = T \times (\mathbb{B} \rightarrow_{\text{fin}} \text{Node}(T))$, so that every node carries exactly one own value and a finite map of children; where absence is wanted, it is a choice of slot. It fixes the structural half of a data floor — shape, keys, paths, canonical order — and commits to nothing about what a tree holds. Construction and navigation need only a carrier; identity additionally uses a supplied equivalence. This paper gives the theory the specification rests on. We define the tree as the initial algebra of a container functor, lift a slot equality to a node identity and prove the lifting a congruence, characterize identity by pointing (paths and the values at them), and give resolution as a partial action of the free monoid of keys. We prove that a node is exactly its parts, and that every node can gain a child under a fresh key while every existing path and value is kept. The model of this paper’s earlier drafts, in which a node either holds a value or has children, embeds into the optional instantiation $\text{Node}(\text{Option}(T))$, and for any inhabited slot no map in the other direction preserves paths and own values. We isolate the admission conditions a slot must meet per capability tier, proving that a byte-valued complete invariant exists iff the slot quotient is countable, and that existence and computability come apart. We prove the three laws of the positional spelling κ (injective, prefix-free, order-preserving), from which sequence semantics embed into the key space at zero interpretive cost. We then develop the reading principle: collection and positional semantics (order, sets, multiplicity, records) enter as *structure profiles* — embedding/retraction pairs into the key space — rather than as constructors or type parameters, and we prove the transport laws for the sequence and set profiles at their stated scope. Finally we exhibit the ontos value model as an exact profile of $\text{Node}(\text{Option}(\mathbb{B}))$. Its mathematical projection and recognizer extend the earlier bridge; the shipping bridge still targets the previous core API.

Scope of draft v4 (2026-09-23). This paper states ADR 0010’s mandatory-value floor, $\text{Node}(T) = T \times (\mathbb{B} \rightarrow_{\text{fin}} \text{Node}(T))$. The previous optional floor is its explicit instantiation at $\text{Option}(T)$; the earlier leaf/structure model embeds into that instantiation (Remark 2.5). The proofs are mathematical arguments, not proof-assistant verification. The encoder in §7 is an abstract construction demonstrating the lifted invariant law, not a specification of published bytes. The candidate `CODEC.md` now states `deixis-codec-v2`, a grammar for this model (ADR 0011); its proofs, vectors and freeze obligations are separate from this paper. Consumer implementations are not certified by these structural proofs.

1 Introduction

Mathematical scope and review. The aim is expressive power through a small, coherent set of concepts. Defined objects have their defining properties; constructions and derived laws are assessed within their stated domains and assumptions. There is no failure until the model defines a failure concept. Partial selection and expressly defined recognizer refusals remain part of the mathematics; physical contingencies enter only through an explicit model that introduces them. A calculator losing power does not qualify $2 + 2 = 4$, and an unmodeled stuck tape does not qualify a Turing machine’s transition rule.

Review this paper for internal consistency, precise definitions, well-defined constructions, valid derivations, expressive reach, and economy of concepts. A defining law states what an admitted object is; a claim that a particular construction supplies such an object still needs justification. Counterexamples must satisfy the stated premises. Do not add feedback about phenomena the model does not contain, or make implementation a prerequisite for theoretical value. Constructions, embeddings, and composition results can establish that value mathematically. Separate implementation claims concern whether particular artifacts realize the model. Draft v4.1 adds this methodological clarification to v4 without changing its definitions or theorems; the paper’s README states the common scope.

Systems that let independent parties agree without trusting each other need, at the bottom, a notion of *value* whose identity can be decided by looking — no context, no clock, no authority. Every layer above such a floor (canonical encodings, content addressing, re-checkable derivations, typed contracts, revocable mandates) inherits its identity from below and can weaken but never repair it. The floor is therefore where stipulation is cheapest and where every avoidable commitment is dearest: a constructor added to the floor is inherited by everyone forever; a meaning attached to the floor is an opinion everyone must share.

deixis is a floor designed under that discipline, one level beneath a byte-atom value model in production use (ontos, §10). Its design bets are:

1. **One container, closed.** A node is an own value and a finite map from byte keys to nodes. There is one constructor and no node kind, no edge metadata beyond the key itself, no ordered sibling list — and the collection and positional readings a consumer reaches for are recovered as proved embeddings (§9; the general reading principle is stated there with its current scope, not as a universality theorem).
2. **One hole, minimal interface.** The slot, the own value every node carries, admits any carrier whatsoever, an optional one included; deixis interacts with it through a single supplied relation, and each further capability (decidability, canonical bytes, decodability) is priced as one further supplied operation (§6).
3. **Keys are bytes, not a parameter.** Byte keys discharge, once and for everyone, the two obligations any key sort owes: an exact decidable equality (for identity) and a total order (for canonicalization). This is an engineering economy rather than a mathematical necessity (Remark 2.6); what keys *mean* is left to readings above the floor (§8).
4. **Structural identity is fixed; its value clause is supplied.** deixis fixes exact key domains and pointwise recursion, and parameterizes exactly the comparison of own values by the supplied equivalence. That no layer above may coarsen or refine the result is a

protocol rule the ecosystem enforces on its observers (Remark 3.3), not a property of the raw type.

The contributions of this paper are the proofs: that the lifting is a congruence (Thm. 3.2, 3.5); that identity is exactly co-pointing (Thm. 3.13); that resolution is a partial monoid action respecting identity (Prop. 4.2); that a node is exactly its parts and that split and plug are inverse (Prop. 5.1, 5.3); that fresh attachment is exact and available at every node (Thm. 5.6, Cor. 5.7), and that for an inhabited slot no path- and value-preserving map back to the previous model exists (Prop. 5.8); the countability boundary for complete invariants and its computability gap (Thm. 6.1, Prop. 6.2); the correctness of the canonical encoding combinator (Thm. 7.4); the three κ laws (Thm. 8.2); the transport laws for the sequence and set profiles (Prop. 9.3, 9.4); and the exactness of the ontos projection (Thm. 10.1).

2 The tree

Metatheory. The semantic results of this paper (§2–§9, §10) are stated in ordinary classical set theory: sets, quotients, and setoids as carriers-with-equivalence, with no constructive or computability commitments. Statements about *effectiveness* are separate and explicit: they are read in ordinary computability over $\mathbb{N}$ and $\mathbb{B}$, algorithms consume the named representations (byte strings; program indices where indicated), and partiality conventions are stated where used (Prop. 6.2 spells out its two readings of “computable on T ”). No represented-space machinery is invoked, and no claim in this paper depends on one.

Fix $\mathbb{B}$, the set of finite octet strings, with decidable equality and the total unsigned-lexicographic order $<_{\text{lex}}$. For sets K, X write $K \rightarrow_{\text{fin}} X$ for the set of finite partial maps $K \rightharpoonup X$.

Write $\text{Option}(T) = \mathbf{1} \sqcup T$, with elements None and $\text{Some}(x)$ for $x \in T$. It is not part of the floor. It is one choice of slot (Remark 2.4), used below for the previous model and for the container profiles.

Definition 2.1 (The functor). For a set T , let $F_T(X) = T \times (\mathbb{B} \rightarrow_{\text{fin}} X)$.

Since $\mathbb{B} \rightarrow_{\text{fin}} X \cong \sum_{K \in \mathcal{P}_{\text{fin}}(\mathbb{B})} X^K$, $F_T(X) \cong \sum_{(o, K) \in T \times \mathcal{P}_{\text{fin}}(\mathbb{B})} X^K$ over $(o, K) \in T \times \mathcal{P}_{\text{fin}}(\mathbb{B})$: F_T is a container (polynomial) functor with shapes $T \times \mathcal{P}_{\text{fin}}(\mathbb{B})$ and positions K at shape (o, K) . Its initial algebra exists and is the corresponding W-type [2, 3].

Definition 2.2 (Nodes). $\text{Node}(T)$ is the initial F_T -algebra. We write its single constructor $\text{Node} : T \times (\mathbb{B} \rightarrow_{\text{fin}} \text{Node}(T)) \rightarrow \text{Node}(T)$. In $\text{Node}(o, m)$, $o \in T$ is the node’s *own value* and m its *children*.

Every node carries exactly one own value and a finite map of children, which may be empty. A node with children keeps its own value, and a node without children still has one. There are no node kinds: “leaf” names a node without children, which is a fact about shape and says nothing about the value. A missing node is a result of navigation (§4), never a special value of T .

Initiality gives the two structural facts the specification stipulates rather than derives, and it is worth recording why stipulation is nevertheless the honest presentation: the specification addresses implementations in languages whose type systems do not exhibit initial algebras, so the floor asserts the consequences directly and this paper certifies that a model exists in which all of them hold.

Stipulation 2.3 (Structural facts). (i) *Injectivity*: Node is injective. (ii) *Finiteness and well-foundedness*: every node is a finite tree; there is no infinite descent through children, and structural induction and recursion are valid.

Both hold in the initial algebra. The initial algebra’s structure map is injective by Lambek’s lemma, and W-types are well-founded by construction. Note what is *not* present: no node kinds, no node metadata beyond the own value, no edge labels, and no ordering among siblings. The only degrees of freedom of a node are its own value, its key set, and its children.

Remark 2.4 (Optionality is a slot choice). The floor imposes no absence convention. A consumer that wants one instantiates the slot with it. $\text{Node}(\text{Option}(A))$ carries an explicit None or $\text{Some}(a)$ at every node, and under the usual tag-respecting equality $\text{None} \neq \text{Some}(a)$ even when a is “empty”. That inequality belongs to the slot and is declared by its supplier; the floor does not stipulate it. $\text{Node}(\mathbf{1})$ is pure keyed shape. $\text{Node}(\mathbf{0})$ is empty, since every tree has a root value. In every instantiation a sentinel inside the slot is a value at an existing node, never the absence of a node.

Remark 2.5 (The previous model). Earlier drafts of this paper took $F_T(X) = T \sqcup (\mathbb{B} \rightarrow_{\text{fin}} X)$, with constructors Leaf and Struct ; write $L(T)$ for its initial algebra. It embeds into this model at the slot $\text{Option}(T)$, not at T itself, by $E : L(T) \rightarrow \text{Node}(\text{Option}(T))$ with

$$E(\text{Leaf}(x)) = \text{Node}(\text{Some}(x), \emptyset), \quad E(\text{Struct}(m)) = \text{Node}(\text{None}, k \mapsto E(m(k))).$$

The image of E is exactly the trees over $\text{Option}(T)$ in which Some occurs only at nodes without children. That was the previous model’s one restriction: a node could not hold a value and have children. No embedding is promised at an unchanged slot T . An intermediate design (ADR 0009 of the specification) made $\text{Option}(T) \times (\mathbb{B} \rightarrow_{\text{fin}} X)$ the floor itself; that model is exactly $\text{Node}(\text{Option}(T))$, without the restriction. In the previous model a node holding a value could gain a child only by being replaced, which moved its value to another path.

Remark 2.6 (Keys are not a parameter). A two-parameter floor $\text{Node}[K, T]$ was considered and rejected in the specification’s first design record. The honest form of the argument is an obligation count, not an impossibility: a key sort K could, like the slot, owe only an equality at the identity tier and a lawful encoder at the canonicalization tier — the encoder’s bytes would then *supply* the total order. What fixing $K = \mathbb{B}$ buys is that both obligations are discharged once, by the type itself, for every consumer forever, and the one-hole notation survives. An engineering economy, deliberately taken; §8 shows nothing positional is lost.

The total order also does quiet foundational work. With a decidable total order, $\mathcal{P}_{\text{fin}}(\mathbb{B})$ has canonical representatives — strictly sorted, duplicate-free entry lists — so the branching functor is a *strict* polynomial functor rather than a quotient of one: $\text{Node}(T)$ mechanizes as a plain inductive type carrying a decidable, hence proof-irrelevant, sortedness invariant, and no *structural* setoid arises before the slot setoid is even in play. (This is exactly the shipped representation: sorted-unique entry vectors.) For an arbitrary key sort with mere decidable equality, finite maps are a quotient of entry lists up to reordering, and every mechanization would inherit setoid structure at the structural level.

3 Identity

deixis does not define identity; it lifts one. The slot is not a type but a *setoid*: a pair $(T, \approx)$ of a carrier and a relation. Nothing else about the slot is ever consulted.

Definition 3.1 (Lifted identity). Given $(T, \approx)$, define $=_{\approx}$ on $\text{Node}(T)$ by structural recursion:

$$\text{Node}(o, m) =_{\approx} \text{Node}(o', n) \iff o \approx o' \wedge \text{dom } m = \text{dom } n \wedge \forall k \in \text{dom } m. m(k) =_{\approx} n(k).$$

Key comparison in $\text{dom } m = \text{dom } n$ is octet equality; the recursion is well-founded by Stipulation 2.3(ii).

Theorem 3.2 (The lifting preserves equivalence). *If $\approx$ is reflexive (resp. symmetric, transitive), so is $=_{\approx}$. In particular if $(T, \approx)$ is a setoid, $(\text{Node}(T), =_{\approx})$ is a setoid.*

Proof. Each property of $=_{\approx}$ separately, by induction on the node, which is one case since there is one constructor. Reflexivity: $\text{Node}(o, m) =_{\approx} \text{Node}(o, m)$ needs $o \approx o$, which is reflexivity of $\approx$, $\text{dom } m = \text{dom } m$, and $m(k) =_{\approx} m(k)$, the last by the inductive hypothesis on the strictly smaller $m(k)$. Symmetry and transitivity are the same induction, using the same property of $\approx$ on own values, symmetry and transitivity of set equality on domains, and the inductive hypothesis pointwise. (The converse direction of the slogan “deixis lifts what it is given and does not repair it” is immediate: if $x \not\approx x$ then $\text{Node}(x, \emptyset) \neq_{\approx} \text{Node}(x, \emptyset)$.) $\square$

Remark 3.3 (The admission rule: three classes of maps). The raw type $\text{Node}(T)$ still distinguishes what $=_{\approx}$ merges: $\text{Node}(x, \emptyset) \neq \text{Node}(y, \emptyset)$ as elements even when $x \approx y$, and an observer that inspects raw slot representations can tell them apart. Every “merged is merged forever” statement in this paper is therefore relative to *extensional* observers — functions that are setoid morphisms, respecting $\approx$ and $=_{\approx}$.

The specification’s rule that no layer may coarsen or refine identity has two halves with *different* enforcers, and an earlier draft of this remark conflated them. The *no-refinement* half is exactly extensionality: a setoid morphism cannot separate what $=_{\approx}$ merges. The *no-coarsening* half is **not** — a constant observer is extensional and maximally coarsening. Coarsening is instead excluded, where it must be, by requiring the map to *reflect* equality as well as preserve it: $f(N) = f(M) \Rightarrow N =_{\approx} M$. The ecosystem accordingly admits three classes of maps, with graded obligations:

- (a) *Observations* — extensional only. A size, a leaf count, a predicate is naturally lossy; nothing downstream treats its output as an identity surrogate.
- (b) *Identity-bearing representations and migrations* — extensional *and* equality-reflecting; equivalently, embeddings of quotient setoids. The lawful-invariant law (1), profile identity transport (Def. 9.1(iii)), and the ontos embedding (Thm. 10.1(iii)) are exactly this two-sided condition. Reflection does not survive post-composition: a pipeline’s membership in this class must be re-established end to end, or composed from certified reflecting stages — the label is a claim about a whole map, never an inheritance from its parts.
- (c) *Recognizers and decoders* — exact partial inverses. Definition 9.1(i),(ii) states their laws; they refuse rather than repair.

Raw observation is not a fourth class but the *boundary* of the taxonomy, and the specification permits it. An own-value accessor may inspect the representative; that is what makes a debugger, a wire dump or a diagnostic possible at all, and a doctrine forbidding it would forbid tooling. What such an observer may not do is launder its result. A quantity that depends on the representative is not a function of the floor value, and the obligation attaches to the

claim rather than to the inspection: a representative-sensitive result may never be presented as value-determined. Where a map is offered as a function of the value it must be $\approx$ -congruent, which places it in class (a); where it is not, it must say which it is. This is the discipline of (c) stated for observers rather than for decoders — the failure mode is not looking, it is looking and then claiming more than was seen.

This is a protocol obligation the ecosystem enforces (and its conformance vectors test), not a property the raw data type can enforce by itself. Every operation defined in this paper is admitted under its class: the observations of §2–4 are extensional (Prop. 4.2(ii) is the case proved for resolution), and the maps this paper treats as identity-bearing are proved to reflect.

Definition 3.4 (Replacement). For a path $p \in \mathbb{B}^*$ (Def. 4.1) with N/p defined, let $N[p \mapsto M]$ be the node obtained by replacing the subtree at p with M , defined by recursion on p : $N[\varepsilon \mapsto M] = M$, and $\text{Node}(o, m)[(k \cdot p) \mapsto M] = \text{Node}(o, m[k \mapsto m(k)[p \mapsto M]])$. Every node on the way down keeps its own value o and its other children; only the subtree at p is replaced, own value included.

Theorem 3.5 (Congruence). *Let $\approx$ be an equivalence. If N/p is defined and $M =_{\approx} M'$, then $N[p \mapsto M] =_{\approx} N[p \mapsto M']$. More generally $=_{\approx}$ is a congruence for Node : if $o \approx o'$, $\text{dom } m = \text{dom } n$ and $m(k) =_{\approx} n(k)$ pointwise then $\text{Node}(o, m) =_{\approx} \text{Node}(o', n)$.*

Proof. The general statement is Definition 3.1 read as an introduction rule. The replacement statement follows by induction on p : for ε it is the hypothesis; for $k \cdot p$, the two results share the own value o of the node being descended through, which is related to itself by reflexivity of $\approx$, and their child maps agree on every key except k , where the inductive hypothesis applies. Reflexivity (Thm. 3.2) covers the untouched entries. $\square$

3.1 Functoriality in the slot

The slot is a parameter, and the parametricity is itself lawful: $\text{Node}(-)$ is a functor, and the identity theory transports along it.

Definition 3.6 (Map). For $f : T \rightarrow U$ define $\text{map } f : \text{Node}(T) \rightarrow \text{Node}(U)$ by structural recursion:

$$\text{map } f (\text{Node}(o, m)) = \text{Node}(f o, k \mapsto \text{map } f (m(k))).$$

Keys are untouched. $\text{map } f$ applies f to the own value of *every* node, and never adds or removes a node. At $T = \text{Option}(A)$ it may therefore change None ; the presence-preserving mapping is the explicit specialization map ($\text{Option.map } g$).

Proposition 3.7 (Functor laws; structure invariance; setoid transport). (i) $\text{map id} = \text{id}$ and $\text{map}(g \circ f) = \text{map } g \circ \text{map } f$. (ii) $\text{Paths}(\text{map } f N) = \text{Paths}(N)$, and resolution commutes: $(\text{map } f N)/p \simeq \text{map } f (N/p)$ (Kleene equality). (iii) If f is a setoid morphism — $x \approx_T y \Rightarrow f(x) \approx_U f(y)$ — then $N =_{\approx_T} M \Rightarrow \text{map } f N =_{\approx_U} \text{map } f M$. Hence $(T, \approx) \mapsto (\text{Node}(T), =_{\approx})$ with $f \mapsto \text{map } f$ is an endofunctor on the category of setoids, of which Theorem 3.2 is the object part.

Proof. All three by structural induction, which has one case. (i) The own values agree because $\text{id } o = o$ and $(g \circ f) o = g(f o)$, and the children agree pointwise by the inductive hypothesis. (ii) map rewrites no key, so path sets coincide; the resolution clause follows by induction on p . (iii) Own values: related by the morphism property. Children: untouched, equal domains and pointwise-related subtrees by the inductive hypothesis — exactly Definition 3.1. $\square$

Corollary 3.8 (Embeddings lift). *Let f be a setoid embedding: it preserves equality ($x \approx_T y \Rightarrow f(x) \approx_U f(y)$) and reflects it ($f(x) \approx_U f(y) \Rightarrow x \approx_T y$) — the identity-bearing class of Remark 3.3. Then $\text{map } f$ is a setoid embedding as well: preservation is Prop. 3.7(iii), and for reflection, by the pointing characterization (Thm. 3.13), $\text{map } f N =_{\approx_U} \text{map } f M$ forces equal path sets (untouched by map) and $\approx_U$ -equal images at every path, which reflection returns to $\approx_T$ -equal own values. Slot-level embeddings lift to node-level embeddings; neither direction alone would suffice for the claim.*

Theorem 3.9 (Lifting commutes with quotienting). *Let $q : T \rightarrow T/\approx$ be the projection onto the quotient setoid (whose equality is discrete equality of classes). Then $\text{map } q$ induces an isomorphism of setoids*

$$\text{Node}(T)/=_{\approx} \cong \text{Node}(T/\approx).$$

Proof. q preserves and reflects equality by definition of the quotient, so $\text{map } q$ is a setoid embedding (Cor. 3.8) and descends to an injection $\text{Node}(T)/=_{\approx} \hookrightarrow \text{Node}(T/\approx)$. Surjectivity: given $M \in \text{Node}(T/\approx)$, choose a representative for each of its finitely many value classes, one per path — finite choice, a theorem of ZF, within the classical metatheory of §2; at the decodable tier no choice is needed, the exact partial decoder supplying the representative (cf. Prop. 9.4) — and the resulting $N \in \text{Node}(T)$ has $\text{map } q N = M$ by structural induction. $\square$

Remark 3.10 (The protocol stance is the quotient semantics). Theorem 3.9 is the precise sense in which the admission discipline of Remark 3.3 is quotient semantics rather than a substitute for it: the setoid model interprets quotient types [4], and every extensional observer of $\text{Node}(T)$ factors uniquely through $\text{Node}(T/\approx)$. A mechanization in a prover with quotient types can work on the quotient side and transport every result across this isomorphism.

Remark 3.11 (Transport of profiles). A structure profile (§9) is a condition on keys, so $\text{map } f$ preserves a positional image verbatim. A profile whose recognizer consults own values — the set profile’s coherence $k = \text{enc}(x)$ for a member $\text{Node}(\text{Some}(x), \emptyset)$ of its optional carrier — transports along $\text{map}(\text{Option.map } f)$ exactly when the member encoders agree, $\text{enc}_U \circ f = \text{enc}_T$; otherwise the image is honestly left rather than repaired.

3.2 Identity is pointing

Definition 3.12 (Paths of a node). $\text{Paths}(N) \subseteq \mathbb{B}^*$: $\varepsilon \in \text{Paths}(N)$ always, and $k \cdot p \in \text{Paths}(\text{Node}(o, m))$ iff $k \in \text{dom } m$ and $p \in \text{Paths}(m(k))$. Write $\text{val}_N(p) \in T$ for the own value of N/p ; it is defined at every $p \in \text{Paths}(N)$. A path is *terminal* iff N/p has no children, and terminal and interior paths alike carry a value.

Theorem 3.13 (Pointing characterization). *Let $\approx$ be an equivalence. $N =_{\approx} M$ iff (a) $\text{Paths}(N) = \text{Paths}(M)$, and (b) $\text{val}_N(p) \approx \text{val}_M(p)$ for every $p \in \text{Paths}(N)$.*

Proof. Write $N = \text{Node}(o, m)$ and $M = \text{Node}(o', n)$. ($\Rightarrow$) By induction on N . At ε , $o \approx o'$ is (b) at ε . Then $\text{dom } m = \text{dom } n$ makes the length-one prefixes agree, and the inductive hypothesis on each $m(k) =_{\approx} n(k)$ extends (a) and (b) under the prefix k . ($\Leftarrow$) By induction on N . (b) at ε is $o \approx o'$. Next, $k \in \text{dom } m$ iff $k \in \text{Paths}(N)$ iff (by (a)) $k \in \text{Paths}(M)$ iff $k \in \text{dom } n$, so the domains agree. For each shared k , the sets $\{p : k \cdot p \in \text{Paths}(\cdot)\}$ and the restricted (b) are exactly conditions (a) and (b) for $m(k), n(k)$, and the inductive hypothesis yields $m(k) =_{\approx} n(k)$. $\square$

Example 3.14 (Neither condition is redundant). For any $x \in T$, the nodes

$$\text{Node}(x, \{a \mapsto \text{Node}(x, \emptyset)\}) \quad \text{and} \quad \text{Node}(x, \emptyset)$$

carry related values wherever both have a path, but their path sets differ (a is in one and not the other). Conversely, for $x \not\approx y$, $\text{Node}(x, \emptyset)$ and $\text{Node}(y, \emptyset)$ have the same path set $\{\varepsilon\}$ and are separated only by (b). At $T = \text{Option}(A)$ the first pair can carry `None` throughout: a path whose value reads as absence is still a path.

The theorem is the origin of the name: a node is what it points at, and nothing else.

Remark 3.15 (What identity is not). Two non-theorems are design content. $=_{\approx}$ is not *object* identity: no address, history, or sharing enters (sharing of equal subtrees is representation and unobservable). And $=_{\approx}$ is not coarser than the lifting: no equation such as “a one-child node equals its child” or “a node whose value is some empty sentinel equals no node at all” is imposed; each would import a consumer’s theory into the floor.

4 Resolution

Definition 4.1 (Paths and resolution). $\text{Path} = \mathbb{B}^*$, the free monoid under concatenation with unit ε . Resolution N/p is the partial function

$$N/\varepsilon = N, \quad \text{Node}(o, m)/(k \cdot p) = m(k)/p \quad (k \in \text{dom } m), \quad \text{undefined otherwise.}$$

Proposition 4.2 (Partial monoid action, respected by identity). (i) $N/(p \cdot q) \simeq (N/p)/q$ (Kleene equality: both sides defined together and equal when defined). (ii) If $N =_{\approx} M$ then for every p : N/p is defined iff M/p is, and $N/p =_{\approx} M/p$ when defined.

Proof. (i) Induction on p . (ii) Definedness of N/p is $p \in \text{Paths}(N)$, invariant by Thm. 3.13(a); the value clause follows by induction on p from Definition 3.1’s pointwise condition. $\square$

Resolution adds no values, constructs nothing on a miss (no autovivification, no default: whatever T holds, including a null-like payload, is the own value of a node that exists, never a stand-in for a node that does not), never enters an own value (it walks children, and only children), and defines no equality. It is a reading of structure that is already there.

5 Composition

A node is exactly its parts, and every existing node can gain a child without anything else changing. This section states the laws that make both precise. They are the reason the floor is a product rather than the previous sum (Remark 2.5). The operations named here are mathematical descriptions, not required methods of an implementation.

Proposition 5.1 (Complete parts). *Let*

$$\begin{aligned} \text{decompose}_T &: \text{Node}(T) \longrightarrow T \times (\mathbb{B} \rightarrow_{\text{fin}} \text{Node}(T)), \\ \text{compose}_T &: T \times (\mathbb{B} \rightarrow_{\text{fin}} \text{Node}(T)) \longrightarrow \text{Node}(T), \end{aligned}$$

with

$$\text{decompose}(\text{Node}(o, m)) = (o, m), \quad \text{compose}((o, m)) = \text{Node}(o, m).$$

Then $\text{compose} \circ \text{decompose} = \text{id}$ and $\text{decompose} \circ \text{compose} = \text{id}$.

Proof. compose is the structure map of the initial algebra, which is an isomorphism by Lambek's lemma, and decompose is its inverse. $\square$

The child map holds whole subtrees, not their own values, and holds every child under its exact key. A parent's own value is an *input* to reconstruction and is never inferred from its children: $\text{Node}(o, m)$ and $\text{Node}(o', m)$ with $o \not\approx o'$ have identical children and are different nodes. So a new parent needs a supplied value: for a generic T nothing can be chosen, while $\text{Node}(\text{Option}(T))$ may supply None and $\text{Node}(\mathbf{1})$ the unit (Remark 2.4).

Definition 5.2 (Contexts, split and plug). Contexts at a path are defined by recursion on the path. The only context at ε is the hole $\square$. A context at $k \cdot p$ is a triple (o, m, C') of an own value, a finite map with $k \notin \text{dom } m$, and a context C' at p . Plugging a node s into a context gives a node:

$$\text{plug}(\square, s) = s, \quad \text{plug}((o, m, C'), s) = \text{Node}(o, m \cup \{k \mapsto \text{plug}(C', s)\}).$$

Splitting is the partial inverse: $\text{split}_\varepsilon(N) = (\square, N)$, and $\text{split}_{k \cdot p}(\text{Node}(o, n)) = ((o, n \setminus k, C'), s)$ where $(C', s) = \text{split}_p(n(k))$ and $n \setminus k$ is n with the key k removed; it is defined when $k \in \text{dom } n$ and the recursive split is. A context retains every ancestor's own value, the keys leading to the hole, and every sibling subtree. Writing $\text{Ctx}_T(p)$ for these contexts makes the types explicit:

$$\begin{aligned} \text{split}_p : \text{Node}(T) &\rightarrow \text{Ctx}_T(p) \times \text{Node}(T), \\ \text{plug}_p : \text{Ctx}_T(p) \times \text{Node}(T) &\rightarrow \text{Node}(T). \end{aligned}$$

The path index supplies the key k in each recursive context layer. The domain of split_p is exactly the nodes in which p exists.

Proposition 5.3 (Split and plug are inverse). *For every N and every $p \in \text{Paths}(N)$, and for every context C at p and every node s ,*

$$\text{plug}(\text{split}_p(N)) = N, \quad \text{split}_p(\text{plug}(C, s)) = (C, s).$$

Consequently, writing $N[p \mapsto s]$ for $\text{plug}(C, s)$ with C the context of N at p , which agrees with Definition 3.4:

$$N[p \mapsto s]/p = s, \quad N[p \mapsto N/p] = N, \quad N[p \mapsto s][p \mapsto t] = N[p \mapsto t].$$

Proof. Both inverse laws by induction on p ; at ε both are the identity. At $k \cdot p$, plugging the split of $\text{Node}(o, n)$ gives $\text{Node}(o, (n \setminus k) \cup \{k \mapsto \text{plug}(\text{split}_p(n(k)))\}) = \text{Node}(o, n)$ by the inductive hypothesis. Conversely, $\text{plug}((o, m, C'), s)$ has k in its domain, removing k gives back m because $k \notin \text{dom } m$, and the inductive hypothesis gives back (C', s) . The replacement laws follow: resolution of $\text{plug}(C, s)$ at p is s by induction on p ; $N[p \mapsto N/p] = \text{plug}(\text{split}_p(N)) = N$; and by the converse law the context of $N[p \mapsto s]$ at p is again C , so replacing there gives $\text{plug}(C, t) = N[p \mapsto t]$. $\square$

The converse law is the one that is easy to omit. Recovering the context from a result must return *the same* context, not merely some context that could have produced the result. The same argument, applied at each path of a finite set $F \subseteq \text{Paths}(N)$ of which no member is a prefix of another, gives complete cuts. Let $\text{Skel}_T(F)$ retain the ancestor values, keys and

all subtrees outside exactly the holes at F ; let $\text{Node}(T)^F$ denote maps with domain exactly F into whole subtrees. Then

$$\begin{aligned}\text{cut}_F &: \text{Node}(T) \rightarrow \text{Skel}_T(F) \times \text{Node}(T)^F, \\ \text{rebuild}_F &: \text{Skel}_T(F) \times \text{Node}(T)^F \rightarrow \text{Node}(T).\end{aligned}$$

Cutting is defined exactly when all paths in F exist. These maps are inverse on their stated domains, including the empty cut and the root-only cut. Different complete cuts of one tree rebuild the same tree. The holes lie in disjoint subtrees, so the order in which they are cut does not matter.

Definition 5.4 (Containment). $A \sqsubseteq B$ iff $\text{Paths}(A) \subseteq \text{Paths}(B)$ and $\text{val}_A(p) \approx \text{val}_B(p)$ for every $p \in \text{Paths}(A)$.

Containment preserves positions, edges and values, not merely a set of values with its topology forgotten. It is reflexive and transitive, and $A \sqsubseteq B \sqsubseteq A$ gives equal path sets and related values, so $A \approx B$ by Theorem 3.13: $\sqsubseteq$ is a partial order up to $\approx$. It needs no order on T beyond $\approx$. At $T = \text{Option}(A)$ with tag-respecting equality it keeps an old None as None; an order in which None may gain a value is a different relation, which belongs to the optional interpretation.

Proposition 5.5 (Compatible overlay). *Assume $\approx$ is an equivalence. Trees A and B have a common upper bound under $\sqsubseteq$ iff*

$$\forall p \in \text{Paths}(A) \cap \text{Paths}(B), \quad \text{val}_A(p) \approx \text{val}_B(p).$$

For such a pair their partial overlay $A \sqcup B$ is the least upper bound, with path set $\text{Paths}(A) \cup \text{Paths}(B)$ and the respective values at those paths, up to $\approx$. Overlay is commutative, idempotent and associative up to $\approx$, with equal definedness for the two associativity expressions.

Proof. A common upper bound relates both values at any shared path to its value there; symmetry and transitivity give the compatibility condition. Conversely, the union of the two finite nonempty prefix-closed path sets is again such a set. Assign the value from A where its path exists, and from B elsewhere. This assignment and path set determine a finite tree, by grouping nonempty paths at their first key. Compatibility makes it an upper bound of both trees. Every common upper bound contains these paths and related values, so it contains their union; this proves leastness. Commutativity and idempotence follow from the union description. Either association of three operands is defined exactly when all three pairs agree on their common paths; when defined, both have the same union of paths and equivalent values. This proves associativity and equal definedness. $\square$

There is no universal root-only identity for arbitrary T : every upper bound must preserve the root value. $\text{Node}(t, \emptyset)$ is neutral within the component of trees with root equivalent to t . A global least tree exists only when $T/\approx$ has exactly one class. In an optional interpretation the different order $\text{None} \leq \text{Some}(t)$ may supply an absence-filling overlay; it is not the relation just proved. Overlay also loses operand provenance: for $e = \text{Node}(t, \emptyset)$ and $s = \text{Node}(t, \{k \mapsto e\})$, $e \sqcup s = s \sqcup e = s \sqcup s$ up to identity. The inverse laws for complete parts do not recover operands of an overlay, edit history, or external effects.

Theorem 5.6 (Exact fresh attachment). *For nodes A, B , a path p and a key k , write $r = p \cdot k$. The operation $\text{attach}(A, p, k, B)$ has as its domain exactly the arguments with $p \in \text{Paths}(A)$ and $r \notin \text{Paths}(A)$; there, with $A/p = \text{Node}(o, m)$,*

$$\text{attach}(A, p, k, B) = A[p \mapsto \text{Node}(o, m \cup \{k \mapsto B\})].$$

The right-hand side is defined on the whole domain (availability), and $C = \text{attach}(A, p, k, B)$ satisfies, exactly,

$$\text{Paths}(C) = \text{Paths}(A) \cup \{r \cdot q : q \in \text{Paths}(B)\},$$

$$\text{val}_C(p') = \text{val}_A(p') \quad (p' \in \text{Paths}(A)), \quad \text{val}_C(r \cdot q) = \text{val}_B(q).$$

Outside the domain, that is when $p \notin \text{Paths}(A)$ or $r \in \text{Paths}(A)$, it is undefined.

Proof. Definedness: A/p is defined because $p \in \text{Paths}(A)$, and $k \notin \text{dom } m$ because $r \notin \text{Paths}(A)$, so $m \cup \{k \mapsto B\}$ is a finite partial map and replacement at p is defined. Undefinedness outside the domain is by definition, not by accident of the formula: an occupied key is refused even when $B = m(k)$, where the union would still be a function. Attachment is not an upsert. Paths and values: replacement keeps the own value and the other children of every node above p (Definition 3.4), and the new node at p keeps o and m and adds exactly the subtree B under k . The new paths $r \cdot q$ cannot coincide with old ones, since an old path extending r would make r old by prefix closure. $\square$

Corollary 5.7 (Consequences of attachment). *In the situation of Theorem 5.6: $A \sqsubseteq C$; $C/r = B$; the own value at p is unchanged; restricting C to the paths of A , keeping those paths and their own values, gives back A , so every old value is retained as it was, at its old path; and attachments at p under distinct fresh keys $k \neq j$ commute: $\text{attach}(\text{attach}(A, p, k, a), p, j, b) = \text{attach}(\text{attach}(A, p, j, b), p, k, a)$.*

Proof. All but the last are read off the exact paths and values in Theorem 5.6. The restriction statement is stronger than $A \sqsubseteq C$, which relates old values only up to $\approx$; attachment keeps them exactly. For commutation, both orders put $\text{Node}(o, m \cup \{k \mapsto a, j \mapsto b\})$ at p and change nothing else. $\square$

Availability is the half that separates this model from the previous one, and a law stated only for successful attachments would not capture it: an operation that refused every childless parent, or that returned A unchanged, would satisfy $A \sqsubseteq C$ vacuously or trivially. Inside the target of E (Remark 2.5), a child can be attached below $\text{Node}(\text{Some}(x), \emptyset)$, the image of $\text{Leaf}(x)$, and the result lies outside the image of E . In the previous model the same node could gain a child only by being replaced, which moved its value to another path.

Proposition 5.8 (No path- and value-preserving map to the previous model). *Let T be inhabited. Read the previous model's observations through E : a previous tree L has the paths of $E(L)$, and the optional value $\text{val}_{E(L)}(p)$ at each of them. Then there is no function $\Phi : \text{Node}(T) \rightarrow L(T)$ with*

$$\text{Paths}(\Phi(N)) = \text{Paths}(N) \quad \text{and} \quad \text{val}_{E(\Phi(N))} = \text{Some} \circ \text{val}_N$$

for every N .

Proof. Take $x \in T$ and $N = \text{Node}(x, \{k \mapsto \text{Node}(x, \emptyset)\})$. If Φ existed, $E(\Phi(N))$ would have the value $\text{Some}(x)$ at ε and the path set $\{\varepsilon, k\}$. A previous tree with a value at ε is $\text{Leaf}(x')$ for some x' , and $E(\text{Leaf}(x')) = \text{Node}(\text{Some}(x'), \emptyset)$ has the path set $\{\varepsilon\}$. So no previous tree has both observations, and $\Phi(N)$ cannot exist. $\square$

The statement is about the required interface, not about bijections between underlying sets: a lossless profile of this model inside the previous one exists, at the cost of changed native paths. The design records give this argument informally; the proposition makes it formal, and slightly stronger, since it rules out every function with those properties and not only isomorphisms.

6 Slots: admission and its tiers

Every requirement deixis places on a slot is a requirement on the pair $(T, \approx)$, never on the carrier alone. The capabilities form a ladder of tiers — and, answering a gap between the specification’s motivation (“identity decided by looking”) and its formal minimum (an equivalence), the effective tiers are now explicit:

supplied	available	
carrier T	shape, keys, paths, navigation	§2–4
+ $\approx$ an equivalence	<i>semantic</i> identity $=_{\approx}$	§3
+ a decision procedure for $\approx$	<i>effective</i> identity	Prop. 6.3
+ $\text{enc} : T \rightarrow \mathbb{B}$ lawful, computable	canonical bytes, hashing, addressing	§7
+ an exact partial inverse for enc	decodable canonical representation	§7

where *lawful* means the complete-invariant law

$$x \approx y \iff \text{enc}(x) = \text{enc}(y), \quad (1)$$

i.e. enc factors through the quotient $T/\approx$ and is injective on it. One inclusion between the tiers is a theorem: a computable lawful enc automatically supplies a decision procedure for $\approx$ (compute and compare), so the canonical-bytes tier contains the effective tier. We claim *no* strictness in the other direction. Prop. 6.2 separates a different pair of notions — set-theoretic existence from computable existence, over a slot whose $\approx$ is undecidable — and says nothing about whether decidable equality falls short of a computable invariant. Indeed, in natural effective presentations the gap can close outright: on $\mathbb{N}$ with any decidable equivalence E , the least-representative map $c(n) = \min\{m \leq n : m E n\}$ is a computable complete invariant. Whether some fixed effective model makes the separation strict is a question about that model’s definition of “computable on T ,” and this paper does not pose one. A slot admitted only at the semantic tier remains mathematically legitimate and operationally mute: its node identity is well-defined and undecidable.

Theorem 6.1 (Existence boundary, non-effective). *A function enc satisfying (1) exists (as a set-theoretic function, with no computability claim) iff $T/\approx$ is countable.*

Proof. $\mathbb{B}$ is countably infinite. A lawful enc induces an injection $T/\approx \hookrightarrow \mathbb{B}$, forcing countability; conversely any injection of the quotient into $\mathbb{B}$, composed with the projection, is lawful. $\square$

Proposition 6.2 (Existence and computability come apart). *Let $T \subseteq \mathbb{N}$ be the set of indices of total computable functions $\mathbb{N} \rightarrow \mathbb{N}$ and $i \approx j$ iff $\varphi_i = \varphi_j$ extensionally. Then $T/\approx$ is countable, so a lawful enc exists (Thm. 6.1); but there is no computable procedure that halts on every $i \in T$ and is lawful there — neither a total computable function on all indices whose restriction to T is lawful, nor a promise procedure guaranteed to halt on the promise T . (Since T is not decidable, “computable on T ” must be one of these two readings; the proof covers both.)*

Proof. Countability is immediate. Suppose enc is computable in either reading and lawful on T . Fix an index $z \in T$ of the constant-zero function. Given any program e and input x , the s-m-n theorem [5] effectively yields an index $g(e, x)$ of

$$f_{e,x}(n) = \begin{cases} 1, & \text{if } e(x) \text{ halts within } n \text{ steps,} \\ 0, & \text{otherwise,} \end{cases}$$

which is total for *every* e, x — so $g(e, x) \in T$ always, and the promise is met. Now $f_{e,x} = \varphi_z$ (extensionally) iff $e(x)$ never halts. Computing and comparing $\text{enc}(g(e, x))$ and $\text{enc}(z)$ — both calls halt, since both indices satisfy the promise — decides, by lawfulness on T , whether $g(e, x) \approx z$, hence whether $e(x)$ halts. That decides the halting problem; contradiction. $\square$

Proposition 6.3 (Decidability transports). *$=_{\approx}$ is decidable iff $\approx$ is. (From $\approx$ to $=_{\approx}$ by structural recursion: key comparison and domain comparison are decidable outright, and own values are compared by $\approx$. Conversely, $x \approx y$ iff $\text{Node}(x, \emptyset) =_{\approx} \text{Node}(y, \emptyset)$.)*

Two consequences frame the design space. Since every carrier admits at least the discrete and the total relation, *no carrier is inadmissible*; only relations are (a non-equivalence lifts to a non-equivalence, Thm. 3.2). And since whatever $\approx$ merges is merged for every layer above — Prop. 4.2(ii) shows no *extensional* observation through the structure separates merged values, and Remark 3.3’s admission classes ensure every admitted map is at least extensional — the choice of $\approx$ is unrecoverable in the disciplined ecosystem, which is why the specification counsels choosing it as fine as any consumer will need.

7 Canonical bytes: the codec combinator

deixis owns only half an encoding: the structural half (tags, counts, key bytes, entry order — discharged by $\mathbb{B}$ and $<_{\text{lex}}$); the slot half is the supplied enc . What the floor can define is a combinator.

Definition 7.1 (Framing assumptions). Fix a length code $\langle \cdot \rangle : \mathbb{N} \rightarrow \mathbb{B}$ that is *injective, prefix-free*, and equipped with a unique exact partial decoder — for every byte stream, at most one prefix is in the code’s image, and the decoder recovers its unique preimage and rejects everything else, including non-canonical spellings of a length (e.g. shortest-form unsigned LEB128 with non-shortest forms rejected) — and a fixed marker string τ , which may be empty.

Definition 7.2 (The combinator). Given lawful enc , define $\text{enc}_D : \text{Node}(T) \rightarrow \mathbb{B}$ by structural recursion:

$$\begin{aligned} \text{enc}_D(\text{Node}(o, m)) &= \tau \cdot \langle |\text{enc}(o)| \rangle \cdot \text{enc}(o) \cdot \langle |m| \rangle \\ &\quad \cdot \prod_{\substack{<_{\text{lex}}\text{-ascending} \\ k \in \text{dom } m}} \langle |k| \rangle \cdot k \cdot \text{enc}_D(m(k)) \end{aligned}$$

where the product concatenates entries in strictly $<_{\text{lex}}$ -ascending key order (keys are unique, so the order is total and the sequence strict). This is an abstract encoder family over the mandatory-value model. ADR 0011 of the specification selects τ empty for `deixis-codec-v2`; that codec’s contract, not this paper, fixes the byte layout, identifier, linked form and refusal protocol. Nothing below depends on those choices, or on the length of τ .

Lemma 7.3 (Self-delimitation). *enc_D is a prefix code: for all N, M , if $\text{enc}_D(N)$ is a prefix of $\text{enc}_D(M)$ then $\text{enc}_D(N) = \text{enc}_D(M)$; moreover a byte stream determines its parse as (tag, framed fields) uniquely.*

Proof. By induction on nodes, using prefix-freeness of $\langle \cdot \rangle$. After the fixed marker, the own-value length determines that payload’s extent. The child count fixes how many entries follow; each key-length field fixes its key’s extent, and the inductive hypothesis fixes its child’s extent. This determines a unique end for the whole node, including one with no children, so no complete encoding can be a proper prefix of another. $\square$

Theorem 7.4 (Lifted codec law). *Let enc be lawful for the slot setoid $(T, \approx)$. Its lifted encoder is lawful for nodes:*

$$N \approx M \iff \text{enc}_D(N) = \text{enc}_D(M).$$

Proof. $(\Rightarrow)$ Induction. Related own values encode equally by (1). Equal domains yield the identical $<_{\text{lex}}$ -sorted key sequence; pointwise $\approx$ children encode equally by the inductive hypothesis; the concatenations coincide. $(\Leftarrow)$ Equal byte strings parse identically (Lemma 7.3): equal own-value payloads give related values by (1); equal counts, then entry-by-entry equal keys and equal child encodings, giving equal domains and, by the inductive hypothesis, pointwise $\approx$ children — exactly Definition 3.1. $\square$

Corollary 7.5 (Content addressing, computational). *Let H be a digest. Unconditionally, $N \approx M \Rightarrow H(\text{enc}_D(N)) = H(\text{enc}_D(M))$: equal nodes always share an address. The converse is not a mathematical fact — no fixed-size digest is injective on an infinite domain — but a computational one: under a collision-resistance assumption on H , exhibiting $N \not\approx M$ with $H(\text{enc}_D(N)) = H(\text{enc}_D(M))$ is infeasible. Address equality is therefore evidence, not proof, of node identity: retrieved content must be rehashed against its address, and where identity is load-bearing, its canonical bytes revalidated. With that discipline, hash-keyed leaves recover shared substructure and DAGs without a cycle ever entering a value.*

Remark 7.6 (Encoder versus codec). What Theorem 7.4 delivers is a computable *complete invariant* — an encoder. A *codec* additionally ships the exact partial decoder: a normative parser that reconstructs the node *up to identity* and rejects, with stable reasons, every byte string outside the canonical image — malformed or non-canonical lengths, malformed slot-codec identifiers, unsorted or duplicate keys, counts inconsistent with the stream, payloads outside the slot codec’s image, trailing bytes, and resource-limit breaches (depth, size) as implementation policy distinct from the value set. The decoder laws are setoid-level, because a lawful enc is injective only on the quotient: $D(\text{enc}_D(N)) \approx N$ (never raw equality of syntax trees — a slot relation coarser than representation makes raw recovery impossible by design), and $\text{enc}_D(D(b)) = b$ byte-exactly on the decoder’s domain. Countability (Thm. 6.1) bounds the existence of *invariants*; an effective codec needs the decodable tier of §6. The specification deliberately freezes no byte layout yet. Its candidate, `deixis-codec-v2` (CODEC.md, ADR 0011),

instantiates this model with one node production — the node’s own payload, length-framed, then its count and entries — and τ empty. Its framing and identity laws are proved separately, in prose, against that concrete grammar (CODEC-proofs.md), and are pinned by independently authored vectors. The earlier leaf/structure grammar survives only as the withdrawn `deixis-codec-v1`. Until the freeze (a clean-room implementation and a signed manifest) the grammar remains a candidate; this theorem does not discharge that delivery gate. A *deployed* content address must additionally bind the codec version, the slot codec identifier, and the digest algorithm (domain separation or a typed envelope) — an interoperability obligation of the frozen artifact, outside this paper’s scope.

8 The positional spelling

Sequence semantics enter deixis by writing positions into keys. The profile `deixis-pos-v1` fixes the spelling once, generically.

Definition 8.1 (mag and κ). For $i \in \mathbb{N}$ let $\text{mag}(i)$ be the shortest unsigned big-endian encoding of i , except $\text{mag}(0) = 00$ (one octet; the only magnitude with a leading zero octet). With $r(i) = |\text{mag}(i)| - 1$:

$$\kappa(i) = \text{ff}^{r(i)} \cdot 00 \cdot \text{mag}(i), \quad |\kappa(i)| = 2|\text{mag}(i)|.$$

The spelling has an honest lineage: κ is a byte-alphabet Elias-gamma code [12] with the unary length prefix written in the *maximal* symbol. Gamma over bits ($0^k 1 \cdot \text{bin}(i)$) is prefix-free but order-reversing under lexicographic comparison, since leading zeros sort low; writing the run in `ff` is exactly the inversion that makes length dominate in the right direction. The construction belongs to the ordered-key family; the contribution here is its ratification as a named, versioned profile with the three laws proved and vector-pinned.

Theorem 8.2 (The three κ laws). (i) κ is injective. (ii) κ is prefix-free: no $\kappa(i)$ is a proper prefix of a $\kappa(j)$. (iii) κ is order-preserving: $i < j \iff \kappa(i) <_{\text{lex}} \kappa(j)$.

Proof. Write $r_i = r(i)$. Observe first that the `ff`-run length of $\kappa(i)$ is exactly r_i : the run is terminated by the `00` octet, and $\text{mag}(i)$ ’s leading octet is `ff` only when it would extend the run — but then the parse below still recovers r_i from the terminator position, which is octet index r_i by construction.¹

(ii) Suppose $r_i < r_j$. At octet index r_i , $\kappa(i)$ carries `00` while $\kappa(j)$ carries `ff` (still inside j ’s run). They differ at an index interior to both, so neither is a prefix of the other. If $r_i = r_j$, the two have equal length $2(r_i+1)$, and a prefix relation forces equality.

(i) If $\kappa(i) = \kappa(j)$ then $r_i = r_j$ (terminator position) and $\text{mag}(i) = \text{mag}(j)$; shortest-form magnitudes of equal length are equal iff the numbers are, and the $\text{mag}(0)$ exception is the unique single-octet magnitude with leading `00`.

(iii) If $r_i < r_j$: numerically, shortest form gives $i < 256^{r_i+1} \leq 256^{r_j} \leq j$ (the middle inequality since $r_i + 1 \leq r_j$; the last since $|\text{mag}(j)| = r_j+1$ forces $j \geq 256^{r_j}$, noting $r_j \geq 1$ here); lexicographically, the first differing octet is index r_i , where `00` < `ff`. So larger run implies both larger number and lexicographically larger key. If $r_i = r_j$: the prefixes agree through

¹Concretely: octets $0, \dots, r_i - 1$ are `ff` and octet r_i is `00`; whether later octets are `ff` is irrelevant to the claims, which only compare octets up to index $\max(r_i, r_j)$ and, on equal r , the magnitudes.

the terminator, and equal-length big-endian magnitude comparison is numeric comparison. Together with (i), trichotomy on both sides yields the equivalence. $\square$

Corollary 8.3 (Sequences at zero interpretive cost). *For a node whose child domain is $\{\kappa(0), \dots, \kappa(n-1)\}$, canonical ($<_{\text{lex}}$ -ascending) entry order equals index order. Consequently: a consumer walking a sequence generates $\kappa(0), \kappa(1), \dots$ and compares octets — no key is ever parsed, and no decoder need exist. Membership of an arbitrary byte string in $\text{im } \kappa$ is decidable by the run/terminator/shortest-form checks alone.*

The specification’s recognition discipline — *exact or refusal* — is a corollary of the profile laws, not of extensionality alone: a reader that “repairs” a non-canonical spelling (e.g. reads `ff000001` as position 1) is a recognizer that identifies two nodes $=_{\approx}$ distinguishes — it coarsens identity *through the reading*, breaking exactness and identity transport (Def. 9.1(ii),(iii)). Extensionality would not forbid it (a repairing reader can be perfectly extensional); the identity-bearing obligations of Remark 3.3(b),(c) do — and once such a reading is deployed, no extensional observer downstream could detect or undo the coarsening, which is why the refusal is placed at the recognizer.

Remark 8.4 (Where prefix-freeness earns its keep). The floor’s path type never concatenates keys — a path is a *sequence* of already-delimited keys, and resolution consumes them one at a time — so the sequence embedding below uses only injectivity and order preservation. Prefix-freeness is provided for the *flattened* contexts profiles may define: spelling a whole key path as a single byte string (single-string path names, radix layouts, streamed prefixes), where unambiguous parsing of concatenated keys is exactly the prefix-free property. It costs κ nothing and licenses those readings in advance.

9 Structure profiles: container semantics as embeddings

The branching functor of Def. 2.1 is one point in the space of containers; abstractly one could parameterize it, $\text{Node}(C, T) = \mu X. T \times C(X)$, obligating each C to a lifted congruence and a canonicalization (a *setoid functor*). deixis refuses the parameter: distinct instances would be distinct floors with incomparable identities, and the floor’s value is its uniqueness. Instead:

Definition 9.1 (Structure profile). Let $(A, =_A)$ be the carrier of a reading’s semantics (a setoid). A *structure profile* for A over the slot $(T, \approx)$ is a named, versioned pair

$$P : A \rightarrow \text{Node}(T), \quad R : \text{Node}(T) \rightarrow A$$

of extensional maps satisfying:

- (i) *retraction*: $R(P(a)) =_A a$ for all a ;
- (ii) *exactness*: if $R(n)$ is defined then $P(R(n)) =_{\approx} n$ — recognition inverts onto the same node, so $\text{dom } R$ is, up to $=_{\approx}$, exactly $\text{im } P$;
- (iii) *identity transport*: $a =_A b \iff P(a) =_{\approx} P(b)$ — the reading’s identity reduces to node identity, never extends it;

- (iv) *recognition*: R is extensional ($n \approx n'$ implies equidefinedness and $=_A$ -equal results) and refuses rather than repairs — no normalization of a node outside $\text{im } P$ into it is permitted. Effectiveness of recognition is the *profile's own* obligation, stated relative to the slot's tiers and discharged per profile (the sequence profile by its dense-domain test, the set profile by its coherence assumptions); it does not follow automatically from the slot being effective;

together with the spelling laws its container's semantics require of the keys P writes: *injectivity* always; *order preservation* iff the reading carries order (so canonical entry order restates it); *prefix-freeness* where flattened spellings are intended (Remark 8.4). This is the ontos bridge's (P, R) structure (§10), promoted to the general interface.

A profile declares its carrier. A container node needs an own value like every node, and the floor supplies none (Remark 2.4). The two profiles below take the slot to be $\text{Option}(U)$: None marks the container node, and a node carrying Some is refused as a container. This is the representation the set profile's specification record states and its migrated implementations use. Another carrier would need its own image and recognizer.

Remark 9.2 (Law dependencies; the categorical name). The four laws are deliberately redundant, and the dependency structure matters for conformance. Law (iii) is derivable: the forward direction is extensionality of P ; backward, $P(a) \approx P(b)$ gives $a =_A R(P(a)) =_A R(P(b)) =_A b$ by (iv)-extensionality of R together with (i). The no-repair clause of (iv) restates (ii). A minimal conformance suite therefore generates (i), (ii), and (iv); testing (iii) directly remains useful because vectors cannot observe extensionality itself. Categorically, a profile is a complemented subobject with chosen complement (the refusals) — equivalently a lawful prism $\text{Node}(T) \cong A \sqcup \text{Refused}$ on quotients — and exactness says the idempotent $e = P \circ R$ is a *restriction* idempotent in the sense of restriction categories [6]: not merely $e \circ e = e$ but $e(n) \approx n$ on its domain. A repair pass is precisely a split idempotent that is *not* a restriction idempotent — a retraction that moves points onto the image — which is the one-line categorical form of exact-or-refusal.

Proposition 9.3 (Sequence transport). *Let $T = \text{Option}(U)$ and take $A = \text{Node}(T)^*$ with pointwise-and-length equality. Define $P_{\text{seq}}(N_0, \dots, N_{n-1}) = \text{Node}(\text{None}, \{\kappa(i) \mapsto N_i\})$ and let $R_{\text{seq}}(\text{Node}(o, m))$, for $n = |\text{dom } m|$, be defined iff $o = \text{None}$ and $\text{dom } m = \{\kappa(0), \dots, \kappa(n-1)\}$, returning the children in index order. A node carrying Some is refused whatever its children: accepting it would read two different nodes, differing only in their own values, as the same sequence. Then $(P_{\text{seq}}, R_{\text{seq}})$ satisfies Definition 9.1 (i)–(iv), with identity transport*

$$P_{\text{seq}}(\vec{N}) \approx P_{\text{seq}}(\vec{M}) \iff |\vec{N}| = |\vec{M}| \wedge \forall i. N_i \approx M_i.$$

Proof. κ 's injectivity (Thm. 8.2(i)) makes P_{seq} a function with n distinct keys; the identity equivalence is Definition 3.1 for nodes carrying None plus index-wise key matching. (i) and (ii) are immediate from the own-value and dense-domain conditions; (iv): the own-value condition is a tag test and the domain condition is decided by generate-and-compare (Cor. 8.3), and extensionality holds because $\approx$ -equal nodes have related own values (so the same tag, under the slot's tag-respecting equality), equal domains and pointwise- $\approx$ children, and no non-dense domain is ever read back into a sequence. This is exactly the ontos recognizer of §10, with children left as nodes. $\square$

Proposition 9.4 (Set transport, repaired). *Let $(S, \approx_S)$ be the member sort, enc lawful for it (1), and let $\hat{\mu} : S/\approx_S \rightarrow \text{Node}(T)$ be a member-form assignment — a function on classes (a chosen member form per class; “assignment” rather than “section,” since it is not a right inverse of any given projection), over the slot $T = \text{Option}(U)$. No injectivity of $\hat{\mu}$ is assumed: keys, not children, separate classes, since enc is already injective on the quotient. Take $A = \mathcal{P}_{\text{fin}}(S/\approx_S)$ with extensional equality, and (with enc constant on classes, written $\text{enc}(c)$):*

$$P_{\text{set}}(A) = \text{Node}(\text{None}, \{ \text{enc}(c) \mapsto \hat{\mu}(c) : c \in A \}).$$

This is literally a finite map — one entry per class, keys distinct by injectivity of enc on the quotient. Suppose further the slot supplies decidable coherence: for a key-child pair one can decide whether some (necessarily unique) class c has $\text{enc}(c) = k$ and $\hat{\mu}(c) =_{\approx} \text{child}$ — e.g. via an exact partial decoder for enc (the decodable tier) plus decidable $\approx_S$ and slot equality. Let R_{set} be defined on exactly the nodes carrying None all of whose entries cohere, returning the set of their classes. A node carrying Some is refused, since accepting it would make set identity coarser than node identity. (Deciding coherence needs effective access to the assignment itself — computing $\hat{\mu}(c)$ from a decoded member — not only a decoder for the key invariant. At the decodable tier no choice is involved at all: take $\hat{\mu}(c) := \text{member-form}(D(\text{enc}(c)))$ — the exact partial decoder is the canonical assignment.) Then $(P_{\text{set}}, R_{\text{set}})$ satisfies Definition 9.1 (i)–(iv); membership transports ($c \in A \iff \text{enc}(c) \in \text{dom}$); and insertion is idempotent by construction (a repeated member spells a duplicate key, which the floor refuses to express).

Proof. P_{set} is well-defined as a function into raw nodes precisely because $\hat{\mu}$ is a function on classes — this is the repair: with a member form chosen per class, no ambiguity of raw children arises. (Injectivity of $\hat{\mu}$ plays no role in any clause below.) (iii): equal node identity gives equal domains, hence equal class sets by injectivity of enc on the quotient; conversely equal sets give identical maps. (i): each entry of $P_{\text{set}}(A)$ coheres by construction and R_{set} reads its classes back. (ii): if n carries None and every entry coheres then n and $P_{\text{set}}(R_{\text{set}}(n))$ both carry None, have equal domains and $=_{\approx}$ -equal children (coherence pins each child to $\hat{\mu}(c)$ up to $=_{\approx}$). (iv): coherence is decidable by hypothesis and the own-value condition is a tag test; extensionality follows since coherence respects $=_{\approx}$ of children and $=_{\approx}$ -equal nodes carry the same tag; nothing outside the coherent image is normalized in. $\square$

Remark 9.5 (The setoid-valued variant). The shipping specification of the set profile deliberately does *not* choose representatives: any $\approx_S$ -equal member may sit under the key (“representatives are free”), recognition checks coherence entry by entry, and all laws hold with P_{set} valued in $\text{Node}(T)/=_{\approx}$ rather than in raw nodes. The two presentations agree up to $=_{\approx}$; the assignment is what an implementation that must *produce* a node uses, and the quotient-valued reading is what recognition-first conformance pins. Stating the raw-node version without a chosen assignment — as an earlier draft of this paper did — is a genuine error: distinct representatives yield $=_{\approx}$ -equal but unequal nodes, and the comprehension fails to define a map at all.

Remark 9.6 (Representation scale). Proposition 9.4 is the *self-keyed* presentation, and its laws are exact at every member sort — but as a physical representation it duplicates each member’s complete invariant in the key beside the child. For slot members with bounded canonical bytes this is a compact spelling; for *node* members under $\text{enc} = \text{enc}_D$ it compounds: the key of a set containing S embeds all of $\text{enc}_D(S)$, so singleton nestings $S_{i+1} = \{S_i\}$ double

in size, and no linked representation rescues this — the parent *key* still spells the member’s complete flat encoding. The duplication is intrinsic to exact direct self-keying (an injective key for an arbitrary member must carry enough bytes to distinguish it), not to the transport laws: the same finite sets embed lawfully as a *sorted positional* spelling — dense κ keys with children strictly increasing under a profile-owned total order induced by a complete invariant of member identity, one occurrence per class enforced by strictness — trading one-lookup membership for search. Which representation a profile freezes is an engineering choice the laws do not make; the shipping specification ratifies self-keying for slot members and, for node members, supersedes direct self-keying in favor of the sorted positional shape, gated on the frozen codec.

Remark 9.7 (Scope of the principle). Multiplicity (count-valued maps or a positional embedding) and records (authored name keys) are further profiles of the same shape. But the principle as *proved* extends exactly this far: positional and collection readings — sequences, sets, their composites — enter as profiles; no universality theorem for arbitrary container semantics is claimed. Two obstacles delimit any general statement. A container has *shape* as well as positions: two distinct nullary shapes would both spell the empty map unless a shape marker (a reserved key namespace) is part of the spelling. And the key space is countable, so only countably presented finitary containers can embed uniformly. We conjecture, and leave as future work, the scoped theorem: every countably presented finitary container with decidable shape codes embeds via a shape marker plus per-shape position spellings in disjoint key namespaces; quotient-symmetric collections (sets, bags) are not instances of the raw polynomial frame at all — they are *quotient containers* [10] — and enter, as above, through a complete invariant of member identity where a lawful *enc* exists — as canonical keys (Prop. 9.4) or as a canonical order (Rem. 9.6): an orbit-splitting by a complete invariant, in that vocabulary. Within that scope the moral stands: openness lives in the two dimensions identity does not interpret — the key space and the slot — readings compose in one node under one identity, where parameters would stack functors and fragment the floor.

Remark 9.8 (Signaling and composition — proposed mechanism). How does a consumer learn *which* profile to apply? The floor’s commitment is precise: deixis does not force a profile tag into every payload node — a mandatory in-node tag would leak semantics into identity (two shape-identical payloads with different tags would be different nodes) and would invite exactly the tag-versus-shape repairs the discipline forbids. A node may lie in several profile images at once, and deliberately so — overlap is classification, not ambiguity of identity: a dense- κ node carrying *None* is a sequence and a degenerate record, and generic tooling exploits that. Above the floor, two distinct mechanisms are *proposed* (design in review; labeled as such, not counted among this paper’s results). *Out-of-band claim*: a profile descriptor — named, versioned, hashable — travels beside the payload; it does not alter the payload’s node identity, has its own identity, and is checked by running the named recognizer. *Self-describing wrapper*: a wrapper profile whose reserved key carries the payload’s descriptor by hash, with the payload under another key — the descriptor then *does* enter the wrapper’s structural identity, deliberately, while the enclosed payload retains its own. Descriptor fields, canonical encoding, payload binding, and behavior under an unavailable profile implementation are obligations of that mechanism’s own record. Composition at one structural level is governed by the spelling laws: each profile writes into a declared key region (dense κ -keys; invariant keys; authored-name keys under a shape marker per Remark 9.7), composed profiles must declare disjoint regions, and a composition whose declared regions overlap is refused as ill-formed rather than

resolved by precedence. Version and digest binding for *deployed* claims is the frozen codec artifact’s obligation (Remark 7.6), outside this paper’s scope.

10 The ontos instance

ontos is a value model in production use: $V_o = \mu X. \mathbb{B} \sqcup X^*$, with constructors Atom and Tuple, structural identity $=_o$, and a frozen canonical codec. It predates deixis; the claim proved here is that it is, exactly, a dense-position profile of $\text{Node}(\text{Option}(\mathbb{B}))$. It is the earlier projection followed by E from Remark 2.5.

Instantiate $T = \text{Option}(\mathbb{B})$ with tag-respecting octet equality (so $=_{\approx}$ is absolute). Define the total projection P and partial recognizer R :

$$\begin{aligned} P(\text{Atom } b) &= \text{Node}(\text{Some}(b), \emptyset), \\ P(\text{Tuple}(v_0 \dots v_{n-1})) &= \text{Node}(\text{None}, \{\kappa(i) \mapsto P(v_i)\}), \\ R(\text{Node}(\text{Some}(b), \emptyset)) &= \text{Atom } b, \\ R(\text{Node}(\text{None}, m)) &= \text{Tuple}(R(m(\kappa(0))), \dots, R(m(\kappa(n-1)))), \quad n = |\text{dom } m|. \end{aligned}$$

The tuple clause is defined only when $\text{dom } m = \{\kappa(0), \dots, \kappa(n-1)\}$ and every child recognizes. A node carrying $\text{Some}(b)$ and children refuses. The complete types are $P : V_o \rightarrow \text{Node}(\text{Option}(\mathbb{B}))$ and $R : \text{Node}(\text{Option}(\mathbb{B})) \rightarrow V_o$.

Theorem 10.1 (Exactness of the bridge). (i) $R(P(v)) = v$ for all $v \in V_o$. (ii) $R(d)$ is defined iff $d \in \text{im } P$; and $R(d) = v$ implies $P(v) =_{\approx} d$. (iii) P is an embedding of identities: $v =_o w \iff P(v) =_{\approx} P(w)$. (iv) *Pointing commutes*: the index path $(i_0, \dots, i_k)$ resolves in v iff the key path $(\kappa(i_0), \dots, \kappa(i_k))$ resolves in $P(v)$, to the projection of the same subvalue.

Proof. (i) Induction on v ; the tuple case uses κ ’s injectivity for the domain and Corollary 8.3 to read the children back in index order. (ii) $(\Leftarrow)$ is (i). $(\Rightarrow)$ Induction on d : the None case’s definedness condition forces the domain to be exactly $\{\kappa(0..n-1)\}$ and the children into $\text{im } P$ inductively, so $d = P(R(d))$ up to $=_{\approx}$; at $T = \text{Option}(\mathbb{B})$ with tag-respecting octet equality, $=_{\approx}$ -equal nodes are equal, giving both claims. The Some case requires no children and is the atom clause exactly. (iii) By (i) and functionality of R on its domain: $P(v) =_{\approx} P(w)$ implies (at this slot, equality) $v = R(P(v)) = R(P(w)) = w$; the converse is congruence of the defining recursion. (iv) Induction on the path, unfolding P and Definition 4.1; the step case is exactly the tuple clause of P . $\square$

Corollary 10.2 (Frozen bytes survive the retraction). *With enc_o ontos’ frozen codec,*

$$\text{enc}_o(R(P(v))) = \text{enc}_o(v).$$

This is immediate from Thm. 10.1(i), and worth stating only for what it does and does not say. It says the round trip through deixis is invisible to the incumbent encoding: project, recognize, and every downstream hash and signature over enc_o -bytes is untouched. It does not say $\text{enc}_D(P(v)) = \text{enc}_o(v)$ — no such law is claimed or wanted; the two encoders address different structures (a deixis codec must write keys; ontos’ elides them), and the specification records that their byte strings must differ. ontos bytes are preserved through the retraction into the ontos model, which is the exact guarantee migration needs and the only one on offer.

Two remarks make the correspondence architectural rather than curiosities. First, sibling order: ontos holds order to be part of a tuple, deixis holds order not to be part of a node — and Thm. 8.2(iii) is why there is no conflict: when a key spells a position, the position *is* the key, so the ordering lives in key space and canonical order restates it. Second, recognition never repairs: sparse, shifted, or non-canonically spelled domains are refused, not normalized — by the same identity-coarsening argument as in §8. The *earlier* projection into $L(\mathbb{B})$, its recognizer and laws (i)–(iv) were implemented and *conformance-tested* — hand-authored vectors replayed by independent implementations, including replay of the frozen ontos codec corpus through $R \circ P$; no proof-assistant verification is claimed. The composition with E above is a mathematical transport; this release does not claim that the external bridge has adopted the mandatory-value API.²

11 Related work

Initial-algebra semantics of data types goes back to [1]; W-types are Martin-Löf’s [2], and their presentation via containers/polynomial functors is [3]. Setoids as carriers-with-equality are the standard constructive treatment of quotients [4]. The mathematical ingredients here are individually standard — structural lifting of an equivalence, a recursive path action, injective framed encodings, an elementary countability characterization — and the order-preserving integer trick is folklore in ordered key–value stores; the contribution claimed is the synthesis and its discipline, not the ingredients.

The closest engineered precedents deserve a direct comparison. Deterministic CBOR [16] already specifies self-delimiting values and bitwise ordering of deterministically encoded map keys; IPLD [18] separates a data model (with maps *and* lists as kinds) from named codecs, and IPLD Schemas [19] map richer kinds (structs, unions, enums) onto that model through representation strategies, with Advanced Data Layouts presenting lensed views over generic traversal. deixis differs on four axes. *Identity*: deterministic CBOR provides protocol-selected canonical serialization rules [16], and IPLD’s abstract model leaves canonicalization to specific codecs (DAG-CBOR’s canonical form and rejection requirements, with documented decoder leniencies [21]) — in both, identity is whatever byte equality of a chosen encoding induces; deixis first parameterizes semantic value identity and proves the lift, so equality is defined (and proved a congruence) before any encoding exists, and canonical bytes are a priced tier above it. *Container inventory*: IPLD’s model carries two containers as kinds; deixis ships one and recovers the second (and sets, and maps-with-order) as named profiles with retraction laws — so “is this a list?” is a checkable judgment with a refusal case, not a kind tag. *Recognition*: representation strategies specify decodings without algebraic laws attached; profiles carry retraction, exactness, and identity-transport laws, with hand-authored vectors pinning the refusals. *Migration*: the ontos bridge states and tests retraction laws against an incumbent’s frozen bytes — we claim only what is established here, a proved and vector-tested retraction against one incumbent, and make no assertion about compatibility results neighboring systems may state for theirs. Content addressing over trees and DAGs is Merkle’s [17]; git and IPLD exploit it with fixed vocabularies, where deixis deliberately ships one container and receives the rest as embeddings.

²projection/deixis in the ontos repository; the deixis floor, the positional spelling, and the set profile are conformance-tested across Rust, Go, TypeScript, and Python against shared hand-authored vectors — see *Artifact availability* below. A machine-checked formalization is future work.

The profile interface itself has close formal relatives, and the comparison belongs at the level of laws. A profile’s (P, R) with retraction and exactness is a partial isomorphism in the sense used by invertible syntax descriptions [13]: one total direction, one partial, composing to identity on the recognized image. Ornaments [14] likewise present specialized datatypes as structured views over a common carrier rather than as new types — the same motive as profiles-over-one-floor, pursued through indexed descriptions where deixis uses key spellings. Lenses and prisms [15] organize the same territory around well-behavedness laws; a profile is prism-shaped (build total, match partial), with exact-or-refusal in place of best-effort match and identity transport as an added, load-bearing law (Remark 9.2 makes the restriction-idempotent form precise). The (P, R) -with-laws pattern is also lawful *views* [7, 8]: the answer to “are profiles just views?” is — views, plus transport and refusal laws, plus vectors. The Gentle Art of Levitation [9] is the existence proof for Remark 9.8’s descriptor language living in the substrate it describes, and its hard-won lesson orders the roadmap: the descriptor’s own identity and decoding must be pinned — the frozen codec — before any self-describing wrapper is sound. For the set profile’s effectivity demands, the Kuratowski-versus-Bishop finiteness distinction [11] is exactly why decidable membership and canonical enumeration are required; the listable-and-decidable case is the codec-tier case. IPLD’s Advanced Data Layouts present synthesized views over substrate data [20] and are the nearest engineered relative; profiles differ in demanding the round-trip and transport laws as admission criteria, vectors included, rather than as conventions of a codec implementation.

Artifact availability. Release v0.1.0 (commit 7a1193c) shipped four implementations of the previous leaf/structure core, the positional spelling and the slot-member set profile, with 103 black-box checks per implementation. That is historical evidence for that model. Release v0.2.0 supplies the mandatory-value interfaces in Rust, Go, TypeScript and Python. Its source tag includes this manuscript and the independently authored `mnode-*.json` expectations for the mandatory model, alongside the earlier corpus with its explicit optional-slot interpretation. The evidence ledger, `docs/EVIDENCE.md`, records measured counts and tested revisions. It records codec and downstream adoption status separately.

The harness `tools/conformance/harness.mjs` drives each implementation as a subprocess over NDJSON, sending every case *minus its expected judgment*. No implementation reads the oracle. Continuous integration runs that harness, per-language format, lint, type and unit checks, specification checks, and two compilation passes of this manuscript. Those runs are conformance evidence, not proof. The mathematical derived operations are exercised through the core accessors; they are not all public methods. The external ontos bridge pins the v0.2.0 core and position packages in all three of its languages. Its tests run in ontos’s continuous integration; this repository records that adoption without re-running them.

12 Conclusion

deixis fixes the structural floor — one closed grammar, one open slot, keys as uninterpreted bytes — and this paper has stated and proved the core properties the layers above lean on: lifted identity is a congruence characterized exactly by pointing; resolution is a lawful partial action; a node is exactly its parts, and attachment under a fresh key is exact and available at every node; complete-invariant encoders exist precisely up to countability, computably only strictly less often, and lift through the combinator; positions embed with order preserved by spelling

alone; sequence and set readings enter as profiles with retraction laws, and the general reading principle is stated at its proved scope (Remark 9.7); and the incumbent value floor above is recovered as one dense instance, exactly, its frozen bytes preserved through the retraction. What remains open is recorded rather than implied: a frozen codec with a normative decoder, the scoped profile-universality theorem, and mechanized proofs. The floor’s refusals — no node kinds, no key meanings, no repair, structural identity fixed with only its value clause supplied — are not austerity for its own sake; each is what makes an eternity of the guarantees above affordable. *A node is what it points at* is, in the end, both the definition and the theorem.

References

- [1] J. Goguen, J. Thatcher, E. Wagner. *An initial algebra approach to the specification, correctness and implementation of abstract data types*. In: Current Trends in Programming Methodology IV, 1978.
- [2] P. Martin-Löf. *Intuitionistic Type Theory*. Bibliopolis, 1984.
- [3] M. Abbott, T. Altenkirch, N. Ghani. *Containers: constructing strictly positive types*. Theoretical Computer Science 342(1), 2005.
- [4] M. Hofmann. *Extensional Constructs in Intensional Type Theory*. Springer, 1997.
- [5] R. I. Soare. *Recursively Enumerable Sets and Degrees*. Springer, 1987.
- [6] J. R. B. Cockett, S. Lack. *Restriction categories I: categories of partial maps*. Theoretical Computer Science 270(1–2), 2002.
- [7] P. Wadler. *Views: a way for pattern matching to cohabit with data abstraction*. POPL 1987.
- [8] C. McBride, J. McKinna. *The view from the left*. J. Functional Programming 14(1), 2004.
- [9] J. Chapman, P.-E. Dagand, C. McBride, P. Morris. *The gentle art of levitation*. ICFP 2010.
- [10] M. Abbott, T. Altenkirch, N. Ghani, C. McBride. *Constructing polymorphic programs with quotient types*. MPC 2004.
- [11] D. Frumin, H. Geuvers, L. Gondelman, N. van der Weide. *Finite sets in homotopy type theory*. CPP 2018.
- [12] P. Elias. *Universal codeword sets and representations of the integers*. IEEE Trans. Information Theory 21(2), 1975.
- [13] T. Rendel, K. Ostermann. *Invertible syntax descriptions: unifying parsing and pretty printing*. Haskell Symposium, 2010.
- [14] P.-E. Dagand, C. McBride. *A categorical treatment of ornaments*. LICS 2013; arXiv:1212.3806.
- [15] J. N. Foster, M. B. Greenwald, J. T. Moore, B. C. Pierce, A. Schmitt. *Combinators for bidirectional tree transformations*. ACM TOPLAS 29(3), 2007.
- [16] C. Bormann, P. Hoffman. *Concise Binary Object Representation (CBOR)*. RFC 8949, IETF, December 2020.
- [17] R. Merkle. *A digital signature based on a conventional encryption function*. CRYPTO 1987.

- [18] Protocol Labs. *IPLD documentation: Codecs*. <https://ipld.io/docs/codecs/> (accessed August 2026).
- [19] Protocol Labs. *IPLD Schemas: representation strategies*. <https://ipld.io/docs/schemas/features/representation-strategies/> (accessed August 2026).
- [20] Protocol Labs. *IPLD: Advanced Data Layouts*. <https://ipld.io/docs/advanced-data-layouts/> (accessed August 2026).
- [21] Protocol Labs. *DAG-CBOR codec specification*. <https://ipld.io/specs/codecs/dag-cbor/spec/> (accessed August 2026).